

Software Design Document

Site Doctor

Version 1.0

July 23, 2026

Contents

1	Introduction	3
2	Architecture	3
2.1	Architectural Style	3
2.2	High-Level Architecture Diagram	3
2.3	Node-Level Graph Topology	5
3	Component Design	5
3.1	Component Responsibility Table	5
3.2	Module / Package Layout	6
4	Database Design	6
4.1	Entity-Relationship Diagram	7
4.2	Table Definitions	7
4.2.1	site	7
4.2.2	audit_run	7
4.2.3	issue	7
4.2.4	ux_suggestion	8
4.2.5	fix	8
4.2.6	report	8
4.3	Indexes	9
4.4	Normalization	9
4.5	Constraints	9
5	API Design	9
6	Sequence Diagrams	10
6.1	SEO + UX Combined Run	10
6.2	Fix Approval and Verification Loop	10
7	Deployment Diagram	10
8	Error Handling	11
9	Security Design	12

1 Introduction

This Software Design Document (SDD) describes how the requirements defined in the Site Doctor Software Requirements Specification (SRS v1.0) are realized in software. It covers system architecture, component responsibilities, database design (for the planned persistence layer), API design, key sequence flows, deployment topology, error handling strategy, and security design decisions.

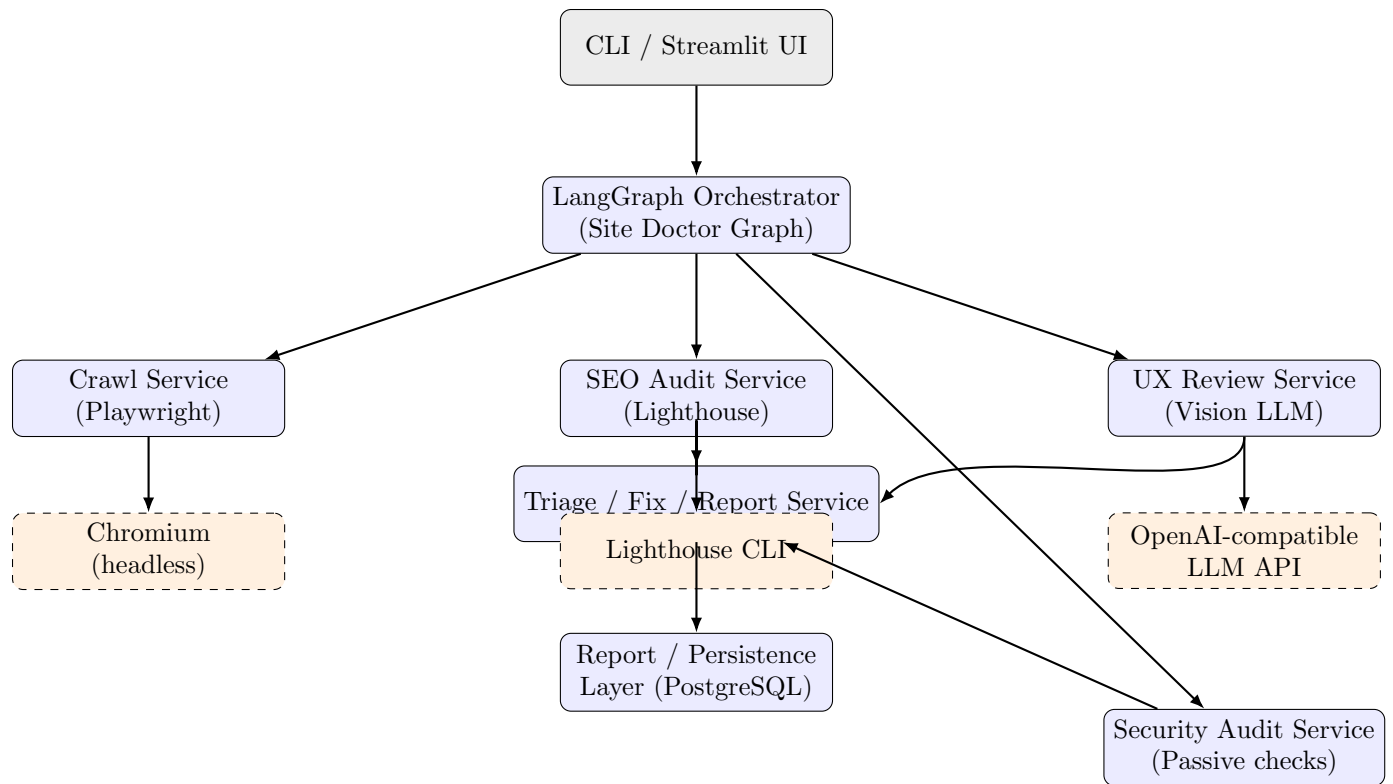
2 Architecture

2.1 Architectural Style

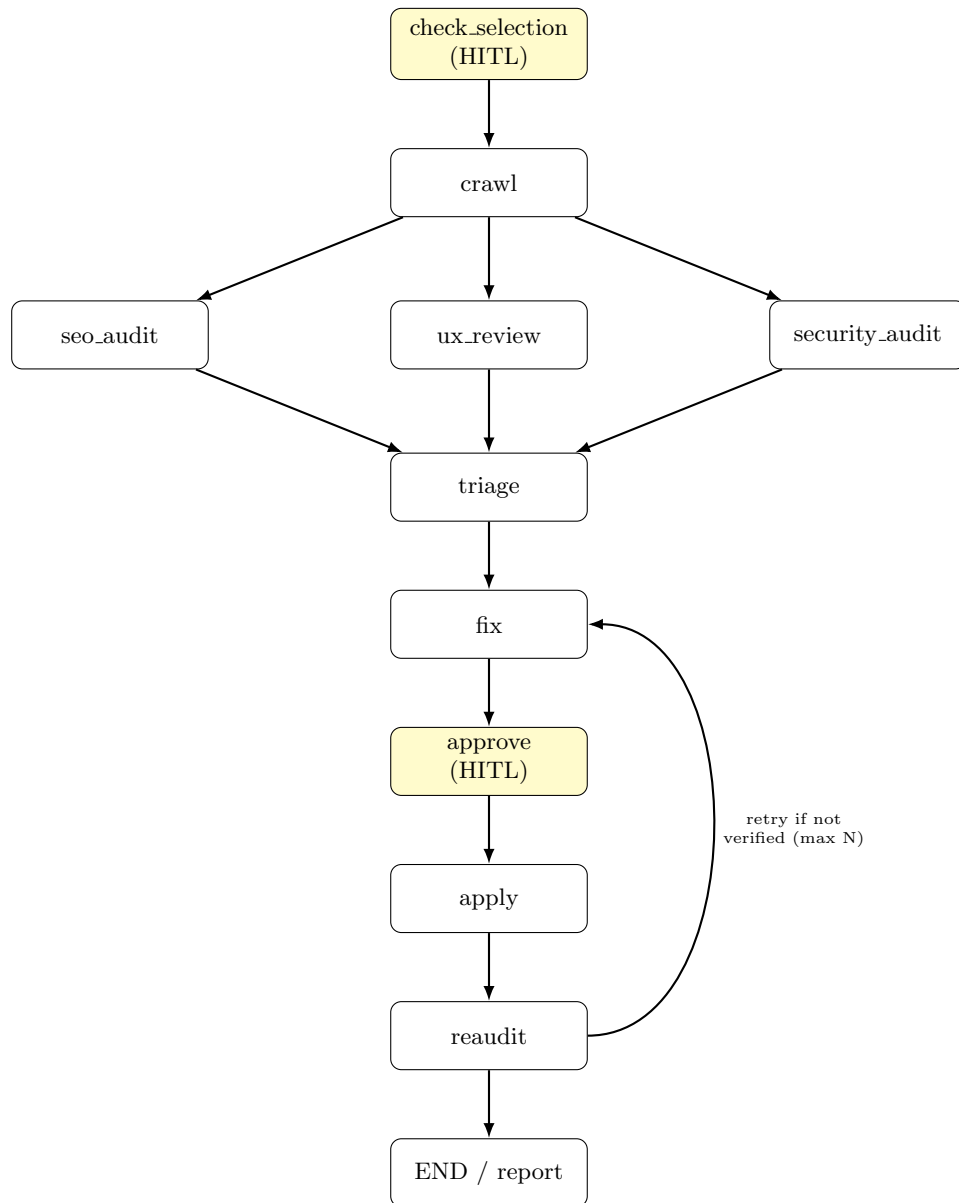
Site Doctor is implemented as a **graph-orchestrated multi-agent pipeline** using LangGraph. Rather than a single linear script, the system is modeled as a directed graph of nodes sharing one typed state object, with three structural patterns used deliberately:

- **Human-in-the-loop interrupt** — the graph pauses after entry to collect the user's check selection before any crawling or audit work begins.
- **Conditional fan-out** — based on the user's selection, the graph dynamically invokes a subset of {seo_audit, ux_review, security_audit} as parallel branches.
- **Fan-in join** — the triage node only executes once every branch that *was* invoked has completed.
- **Act-verify-retry loop** — the fix/approve/apply/reaudit sequence forms a cycle, conditionally routing back to fix-generation when a verification check fails.

2.2 High-Level Architecture Diagram



2.3 Node-Level Graph Topology



Nodes drawn with a yellow fill are explicit human-in-the-loop checkpoints; all others execute autonomously. Note that `seo_audit`, `ux_review`, and `security_audit` are only invoked for branches selected at the `check_selection` step — the diagram shows the full possible topology, not every run’s actual path.

3 Component Design

3.1 Component Responsibility Table

Component	Responsibility
Orchestrator (LangGraph)	Owns the state graph definition, node registration, edges (fixed and conditional), and execution/invoke.
check_selection_node	Human-in-the-loop; collects which checks (SEO/UX/Security) to run for this invocation.
crawl_node	Renders the target page via Playwright; always saves an HTML copy; conditionally captures scroll-position screenshots only if UX is selected.
seo_audit_node	Invokes Lighthouse against the live URL; delegates parsing to the Audit Service.
ux_review_node	Sends the captured screenshot set, in one request, to a vision-capable LLM; delegates parsing to the Vision Review Service.
security_audit_node	Performs passive header/TLS/CVE-lookup checks; only invoked when explicitly selected.
triage_node	Ranks Issues by severity and generates plain-language summaries via LLM call; passes UX Suggestions through unmodified.
fix_node	Generates a proposed Fix (before/after) for high-confidence, mechanically verifiable Issues only.
approve_node	Human-in-the-loop; surfaces proposed Fixes and records per-fix approval/rejection.
apply_node	Writes approved Fixes into the local HTML copy.
reaudit_node	Re-runs the originating audit tool against the patched local copy; marks each Fix verified-cleared or not.
Audit Service (audit/lighthouse.py)	Wraps the Lighthouse CLI subprocess call; parses only tracked audit IDs and category scores out of the raw report before anything is retained in state.
Vision Review Service (ux_review/vision_review.py)	Encodes screenshots, builds the multi-image prompt, calls the LLM, parses structured <code>UXSuggestion</code> objects out of the response.
Crawler Service (crawler/crawl.py)	Wraps Playwright; provides <code>crawl_page</code> (HTML) and <code>screenshot_page</code> (scroll-position screenshots, capped count).

3.2 Module / Package Layout

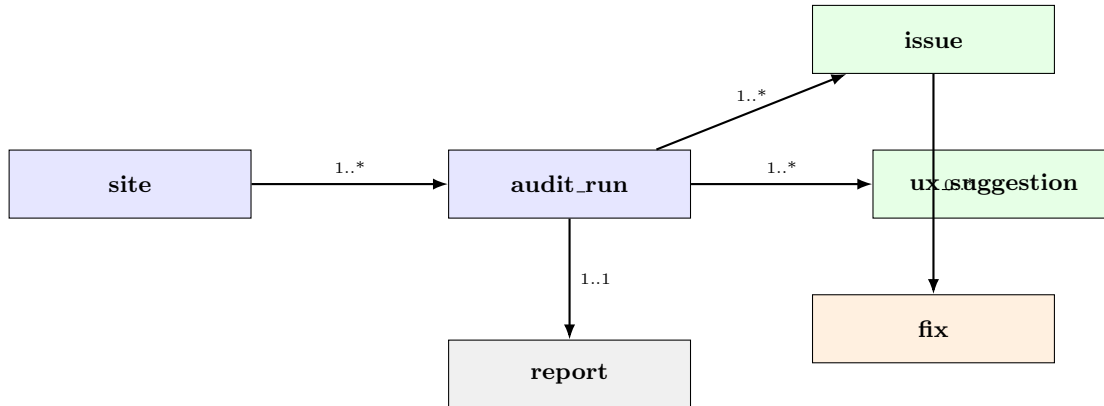
site-doctor/		
agent/	graph.py	LangGraph state machine definition
crawler/	crawl.py	Playwright rendering + screenshots
audit/	lighthouse.py	Lighthouse subprocess + report parsing
ux_review/	vision_review.py	Vision LLM prompt + response parsing
models/	schemas.py	Pydantic schemas (shared state, all cross-node objects)
tests/		Unit tests, one module per component

4 Database Design

The pipeline itself is stateless per run (all state lives in the in-memory LangGraph state object). The persistence layer described below is the planned storage for completed run reports, enabling

report history/download (SRS FR-20) and is deliberately decoupled from the pipeline's internal Pydantic schemas.

4.1 Entity-Relationship Diagram



4.2 Table Definitions

4.2.1 site

Column	Type	Notes
id	UUID (PK)	Primary key.
url	TEXT	Target URL, normalized (scheme + host + path).
created_at	TIMESTAMP TZ	First time this URL was audited.

4.2.2 audit_run

Column	Type	Notes
id	UUID (PK)	Primary key.
site_id	UUID (FK → site.id)	
selected_checks	TEXT[]	e.g. {seo, ux}; security only present if explicitly opted in.
started_at	TIMESTAMP TZ	
completed_at	TIMESTAMP TZ (nullable)	Null while a run is in progress.
status	TEXT	running / completed / failed.

4.2.3 issue

Column	Type	Notes
id	UUID (PK)	
run_id	UUID (FK → audit_run.id)	

Column	Type	Notes
lighthouse_audit_id	TEXT	e.g. <code>image-alt</code> ; stable rule identifier from the tracked-audits map.
category	TEXT	seo / accessibility / performance.
severity	TEXT	high / medium / low, set during triage.
plain_language_summary	TEXT	

4.2.4 ux_suggestion

Column	Type	Notes
id	UUID (PK)	
run_id	UUID (FK → audit_run.id)	
category	TEXT	clutter / cta-overload / hierarchy / trust-signal, etc.
severity	TEXT	high / medium / low.
observation	TEXT	
recommendation	TEXT	

4.2.5 fix

Column	Type	Notes
id	UUID (PK)	
issue_id	UUID (FK → issue.id)	A fix always traces back to a mechanically verifiable Issue, never a UX Suggestion.
before_snippet	TEXT	
after_snippet	TEXT	
approved	BOOLEAN (nullable)	Null until the human decides.
applied	BOOLEAN	
verified_cleared	BOOLEAN (nullable)	Null until reaudit runs.
attempts	INTEGER	Bounded by <code>max_retries_per_fix</code> .

4.2.6 report

Column	Type	Notes
id	UUID (PK)	
run_id	UUID (FK → audit_run.id, unique)	One report per run.
scores_json	JSONB	Category scores at time of report generation.
generated_at	TIMESTAMPTZ	

4.3 Indexes

- `audit_run(site_id)` — supports "history for this site" lookups.
- `issue(run_id)`, `ux_suggestion(run_id)`, `fix(issue_id)` — support the primary fan-out queries when assembling a report.
- `site(url)` — unique index; prevents duplicate site rows for the same normalized URL.

4.4 Normalization

The schema is in Third Normal Form (3NF): every non-key attribute depends only on its table's primary key. `issue` and `ux_suggestion` are kept as separate tables (rather than one polymorphic "finding" table) deliberately, mirroring the SRS/SDD distinction between mechanically verifiable findings and judgment-based ones — this avoids nullable columns that only apply to one finding type and keeps each table's constraints meaningful.

4.5 Constraints

- `fix.issue_id` is NOT NULL — enforces at the schema level that fixes only ever attach to verifiable Issues.
- `audit_run.status` is constrained to the enumerated set {running, completed, failed}.
- `report.run_id` is UNIQUE — one report per run.

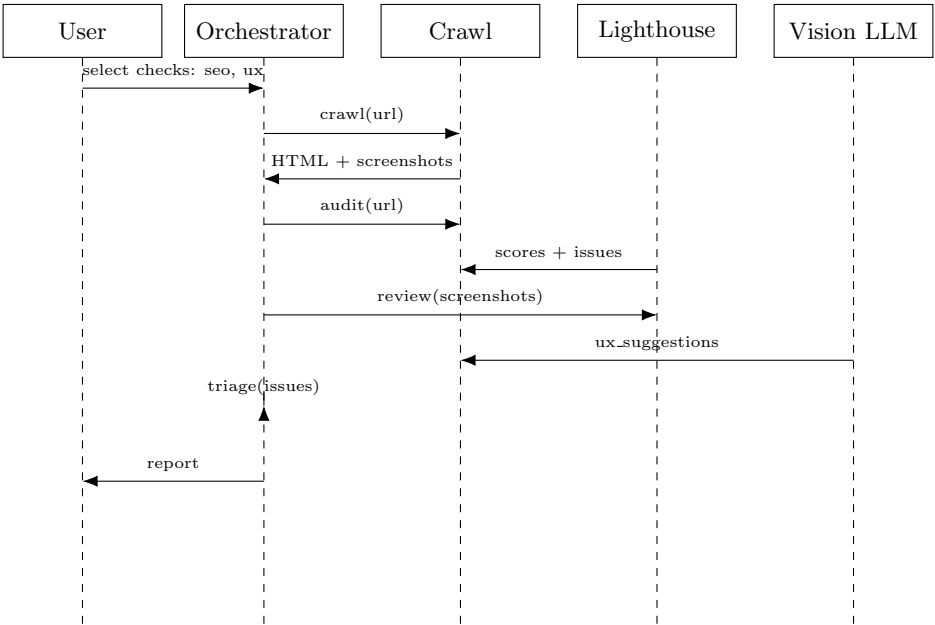
5 API Design

The pipeline is invoked internally as a function call (`graph.invoke(...)`) in its current local-execution form. The table below specifies the external HTTP API surface planned for the hosted/UI-driven version, to keep the SDD forward-compatible with the Streamlit/FastAPI extension mentioned in the SRS.

Method	Path	Description
POST	<code>/api/runs</code>	Start a new audit run; body: {url, selected_checks}.
GET	<code>/api/runs/{id}</code>	Poll run status and, once complete, retrieve the report.
POST	<code>/api/runs/{id}/fixes/{fix_id}/approve</code>	Human approval callback for a proposed fix (used at the approve_node interrupt).
GET	<code>/api/runs/{id}/report</code>	Download the final report (JSON or rendered format).

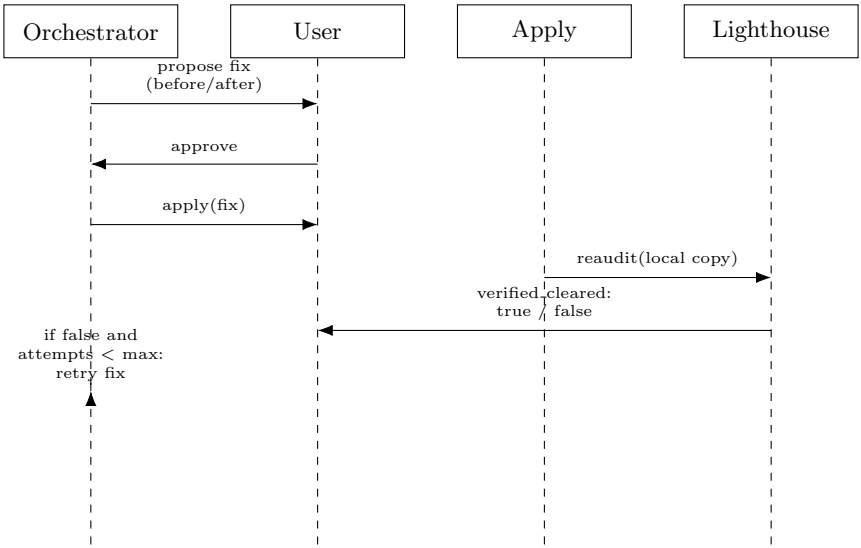
6 Sequence Diagrams

6.1 SEO + UX Combined Run

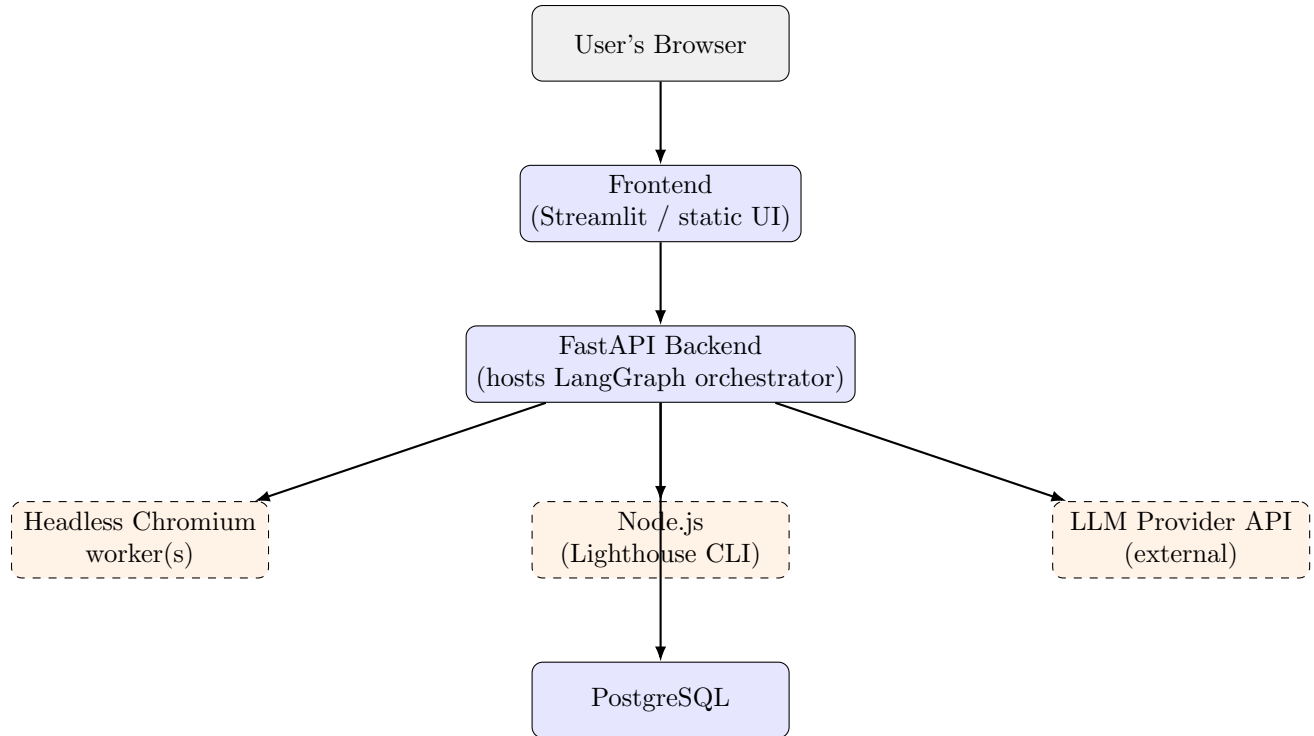


`seo.audit` and `ux.review` branches execute in parallel in the actual implementation; the diagram lists them sequentially for readability, but no ordering dependency exists between them — both depend only on `crawl`’s output.

6.2 Fix Approval and Verification Loop



7 Deployment Diagram



The current implementation runs the API/orchestrator, Chromium, and Lighthouse locally on a single developer machine. The diagram above reflects the target hosted topology; the local-execution version omits the Frontend/FastAPI split and PostgreSQL layer per SRS §2.1.

8 Error Handling

Failure Mode	Handling Strategy
Invalid / unreachable URL	Caught at <code>check_selection</code> → <code>crawl</code> boundary; run terminates with a clear validation error before any external tool invocation (SRS FR-02).
Lighthouse CLI not found on PATH	<code>run_lighthouse</code> falls back to an <code>npx</code> -based invocation; if neither resolves, raises a <code>RuntimeError</code> with an actionable install instruction rather than a raw subprocess traceback.
Lighthouse trace/audit crash (e.g. <code>NO_LCP</code>)	Documented mitigation: use <code>--headless=new</code> Chrome flag; if a category still fails to compute, that category's score/issues are omitted from the report rather than failing the entire run.
Non-UTF-8 file reads on Windows	All file I/O against tool-generated output explicitly specifies <code>encoding="utf-8"</code> rather than relying on platform default encoding.
LLM API failure / time-out	Node-level try/except with a bounded number of retries; on exhausted retries, the affected branch's findings are marked unavailable in the report rather than failing the whole run.

Failure Mode	Handling Strategy
Fix does not verify as cleared	Routed back to <code>fix_node</code> , capped at <code>max_retries_per_fix</code> ; on exhaustion, reported as unresolved with the attempt history retained (SRS FR-26).

9 Security Design

- **Opt-in, off-by-default security auditing.** The `security_audit` branch is only reachable via the conditional fan-out when explicitly selected; this is enforced at the graph-routing level (`route_checks`), not just the UI layer, so no code path can silently enable it.
- **Passive-only scope.** The security module is restricted by design to read-only checks (HTTP headers, TLS configuration, version/CVE lookups). No exploitation, brute-force, or load-generation logic exists anywhere in the module boundary.
- **No live write-back.** `apply_node` writes only to the local, sandboxed HTML copy under `.site-doctor-cache/`; there is no code path that authenticates to or modifies a user's live CMS or hosting infrastructure.
- **Credential handling.** LLM API keys are read from environment variables (`OPENAI_API_KEY`) only; never hard-coded, logged, or included in any report artifact.
- **Transport security.** All external API calls (LLM provider, CVE lookup) occur over HTTPS.